

DOM TRADE

Regime-Aware Forecasting System

Order Book Data Collection & Stock Price Prediction

NSE · Angel One SmartAPI · TimescaleDB · LightGBM · River
FTRL · Hoeffding Tree · PA Classifier · GMM · ARF · ADWIN

Exchange	National Stock Exchange of India (NSE)
Broker API	Angel One SmartAPI (WebSocket Mode 3)
Symbol	RELIANCE-EQ
Training Data	34,743 ticks · May 13 2026 (1 trading day)
Test Data	10,250 ticks · May 14 2026 (out-of-sample)
Models	GMM + ARF + FTRL + Hoeffding + PA
Day 1 Accuracy	85.04% vs 41.15% naive baseline
Day 2 Accuracy	83.41% vs 41.88% naive baseline (generalisation)
Generated	May 2026

Contents

1	Project Overview	4
1.1	What This Project Is	4
1.2	Why Order Book Data and Not Price Alone	4
1.3	Project Architecture	4
1.4	Data	5
2	Feature Engineering	6
2.1	Raw Input Columns	6
2.2	Group 1 — Price Features	6
2.2.1	Mid Price	6
2.2.2	Bid-Ask Spread	6
2.2.3	Volume-Weighted Mid Price (VWMP)	7
2.2.4	LTP–Mid Price Delta	7
2.3	Group 2 — Order Book Features	7
2.3.1	Order Book Imbalance (OBI)	7
2.3.2	Depth Ratio	7
2.3.3	Book Depth / Slope	7
2.3.4	VWAP Pressure	8
2.3.5	Level-1 Concentrations	8
2.3.6	Fill Price (Market Impact)	8
2.4	Group 3 — Support and Resistance Features	8
2.5	Group 4 — Multi-Scale Rolling Features	9
2.6	Feature Summary	9
2.7	Target Engineering	10
3	Regime Detection	11
3.1	Why Regime Detection	11
3.2	Step 1 — Gaussian Mixture Model (GMM)	11
3.2.1	What It Is	11
3.2.2	Why GMM Must Come First	11
3.2.3	Input Features	11
3.2.4	Regime Assignment	11
3.2.5	Why Not HMM Instead of GMM	12
3.3	Step 2 — Adaptive Random Forest (ARF)	12
3.3.1	What It Is	12
3.3.2	Why ARF for Real-Time Regime Detection	12
3.3.3	Input Features for ARF	12
3.3.4	Why Not K-Means Instead of GMM	13

4	Online Forecasting Models	14
4.1	Design Principle	14
4.2	Model 1 — FTRL (Follow The Regularized Leader)	14
4.2.1	Algorithm	14
4.2.2	Why FTRL Over Standard SGD	15
4.2.3	Why FTRL Over Passive-Aggressive for Trending Regimes	15
4.3	Model 2 — Hoeffding Tree (VFDT)	15
4.3.1	Algorithm	15
4.3.2	Why Non-Linear for Regime Boundaries	15
4.3.3	Grace Period Warning	16
4.4	Model 3 — Passive-Aggressive Classifier (PA)	16
4.4.1	Algorithm	16
4.4.2	Why PA for Low-Volatility Regimes	16
4.4.3	Why PA Achieved 96% Accuracy and What That Means	16
4.5	Input Features for All Three Forecasters	17
5	Dynamic Ensemble and Drift Detection	18
5.1	Why Dynamic Weighting and Not Fixed	18
5.2	Two-Stage Weight Computation	18
5.2.1	Stage 1 — Regime Prior	18
5.2.2	Stage 2 — Rolling Accuracy Adjustment	18
5.2.3	Final Prediction	19
5.3	ADWIN Drift Detection	19
5.3.1	What ADWIN Does	19
5.3.2	Why Not Reset Models on Drift	19
5.3.3	On Drift: Boost, Not Reset	19
6	Algorithms Considered But Not Used	20
6.1	Models Evaluated and Rejected	20
6.1.1	Kalman Filter (considered for price regression)	20
6.1.2	LinUCB Contextual Bandit (considered for model selection)	20
6.1.3	HMM (considered for regime detection)	20
6.1.4	DeepLOB / TCN (considered for sequence modeling)	21
6.1.5	LightGBM / XGBoost / CatBoost (tested as offline benchmarks)	21
6.1.6	AROW (Adaptive Regularization of Weights)	21
6.1.7	Ridge / Lasso (tested as price regressors)	22
7	Results	23
7.1	Key Results — Day 1 (May 13 2026)	23
7.2	Per-Regime Accuracy — Day 1	23
7.3	Model Comparison — Day 1	23
7.4	Latency	24
7.5	Drift Events — Day 1	24
7.6	Day 2 Generalisation Test (May 14 2026)	24
8	System Design Decisions	26
8.1	Why Time-Ordered Train/Test Split	26
8.2	Why Binary Target Instead of 3-Class	26
8.3	Why Not Reset Models on Drift	26

8.4	Why Python and Not C++	26
9	Limitations and Future Work	27
9.1	Honest Limitations	27
9.2	Future Work (Priority Order)	27
10	How to Run	29
10.1	Dependencies	29
10.2	Full Pipeline (One Command)	29
10.3	Step by Step	29
10.4	Test on New Day's Data	30
10.5	Dashboard	30
10.6	Live API Streaming	30
10.7	Project File Structure	30

1. Project Overview

1.1 What This Project Is

DOM Trade is an end-to-end adaptive forecasting system for NSE equities built on live Level-2 order book data. The system streams real-time bid-ask depth from Angel One's WebSocket API, engineers 46 microstructure features from raw ticks, detects the current market regime using a two-stage machine learning pipeline, and produces directional predictions (UP/DOWN) using three parallel online models whose outputs are fused by a dynamic weighted ensemble.

The central thesis: A single static model trained once cannot adapt to changing market conditions. Markets behave fundamentally differently when trending up, trending down, or consolidating. This system detects which condition is active and routes predictions through models specifically suited to that regime.

1.2 Why Order Book Data and Not Price Alone

Price alone tells you what happened. The order book tells you what is about to happen. When 50,000 shares are sitting at a bid level, that is a wall — price is unlikely to fall through it easily. When the ask side is thin, price can run up fast. No chart-based or price-only model sees this. Order book microstructure is the closest thing to reading institutional intent in real time.

Key insight: Level-2 data (5 bid levels + 5 ask levels) reveals supply-demand imbalances before they manifest in price. OBI (Order Book Imbalance) is statistically predictive of the next 30-tick mid-price direction with a lead time that allows meaningful action within the WebSocket tick interval.

1.3 Project Architecture

The system is organized into seven layers:

Layer	File	Role
A	<code>feature_engine.py</code>	Computes 46 features from raw ticks
B	<code>regime_detector_v2.py</code>	GMM labels + ARF online detection
C	<code>forecasters.py</code>	FTRL + Hoeffding Tree + PA Classifier
D	<code>ensemble.py</code>	Dynamic weighted fusion + ADWIN drift
E	<code>backtest.py</code>	Full evaluation on held-out test set
F	<code>live_engine.py</code>	Live API streaming and inference

G	<code>dashboard.py</code>	Streamlit visual dashboard
---	---------------------------	----------------------------

Table 1.1: Project layer overview

1.4 Data

Symbol	Company	Token	Rows
RELIANCE-EQ	Reliance Industries	2885	34,743
HDFCBANK-EQ	HDFC Bank	3045	35,934
TCS-EQ	Tata Consultancy Services	11536	27,943
INFY-EQ	Infosys	1594	23,149
ICICIBANK-EQ	ICICI Bank	1660	14,155
Total			135,924

Table 1.2: Dataset composition (2026-05-13, one trading day)

All modelling uses RELIANCE-EQ only (34,743 rows, most liquid name in the set).

2. Feature Engineering

2.1 Raw Input Columns

The Angel One WebSocket Mode 3 delivers 27 columns per tick:

- **Identifiers:** `time`, `symbol`, `token`, `exchange_type`
- **Trade metrics:** `ltp`, `volume` (`avg_price` is always null — dropped)
- **Bid side:** `bid_price_1..5`, `bid_qty_1..5`
- **Ask side:** `ask_price_1..5`, `ask_qty_1..5`

Note

All prices arrive in paise (1 rupee = 100 paise) and are converted to rupees at the `parse_tick()` boundary in the collector layer. By the time any feature is computed, all prices are already in rupees.

2.2 Group 1 — Price Features

2.2.1 Mid Price

$$\text{mid_price} = \frac{\text{ask_price}_1 + \text{bid_price}_1}{2}$$

Why: LTP (last traded price) bounces between bid and ask on every trade and contains noise that is not directionally predictive. Mid price is the true fair value estimate. Predicting LTP movement is predicting noise. Predicting mid price movement is predicting actual market direction. All academic LOB forecasting papers use mid price as the prediction target.

2.2.2 Bid-Ask Spread

$$\begin{aligned}\text{spread} &= \text{ask_price}_1 - \text{bid_price}_1 \\ \text{spread_pct} &= \frac{\text{spread}}{\text{ask_price}_1} \times 100\end{aligned}$$

Why: Tight spread signals a liquid, efficient market where trades happen close to fair value. Wide spread signals uncertainty and potential for large moves. Spread directly determines the transaction cost of any trade.

2.2.3 Volume-Weighted Mid Price (VWMP)

$$\text{vwmp} = \frac{\text{bid_price}_1 \times \text{ask_qty}_1 + \text{ask_price}_1 \times \text{bid_qty}_1}{\text{bid_qty}_1 + \text{ask_qty}_1}$$

Why: Regular mid price treats both sides equally. VWMP weights by quantity — if 10,000 shares want to buy but only 500 want to sell, VWMP skews toward the ask. It is a pressure-adjusted fair value that is more predictive than plain mid price when one side dominates.

2.2.4 LTP–Mid Price Delta

$$\text{ltm_mid_delta} = \text{ltm} - \text{mid_price}$$

Why: When the last trade happened above mid price, it signals buyer aggression (they paid more than fair value). When below mid price, it signals seller aggression. This delta captures the direction of the most recent informed order.

2.3 Group 2 — Order Book Features

2.3.1 Order Book Imbalance (OBI)

Level-1 OBI:

$$\text{obi}_{l1} = \frac{\text{bid_qty}_1 - \text{ask_qty}_1}{\text{bid_qty}_1 + \text{ask_qty}_1} \in [-1, +1]$$

Full 5-level OBI:

$$\text{obi} = \frac{\sum_{i=1}^5 \text{bid_qty}_i - \sum_{i=1}^5 \text{ask_qty}_i}{\sum_{i=1}^5 \text{bid_qty}_i + \sum_{i=1}^5 \text{ask_qty}_i} \in [-1, +1]$$

Why: OBI is the single strongest short-term directional signal in academic LOB literature. When buyers outnumber sellers at the best price, mid price statistically tends to move up. The 5-level version captures deeper structural imbalances that the level-1 version misses. Both versions are retained because they carry different information.

2.3.2 Depth Ratio

$$\text{depth_ratio} = \frac{\sum_{i=1}^5 \text{bid_qty}_i}{\sum_{i=1}^5 \text{ask_qty}_i}$$

Why: Structural buyer-vs-seller dominance across the entire book. Persistent skew above 1 indicates a trending bullish regime. Persistent skew below 1 indicates bearish pressure.

2.3.3 Book Depth / Slope

$$\text{ask_depth} = \text{ask_price}_5 - \text{ask_price}_1$$

$$\text{bid_depth} = \text{bid_price}_1 - \text{bid_price}_5$$

Why: A steep slope means large orders will significantly move price. A flat slope means there is deep liquidity at nearby levels. This directly measures market impact before a trade is placed.

2.3.4 VWAP Pressure

$$\text{ask_vwap} = \frac{\sum_{i=1}^5 \text{ask_price}_i \cdot \text{ask_qty}_i}{\sum_{i=1}^5 \text{ask_qty}_i}$$

$$\text{vwap_pressure} = \frac{\text{ask_vwap} - \text{bid_vwap}}{\text{mid_price}}$$

Why: Captures asymmetry in liquidity depth across all levels, not just at the best bid/ask. A high VWAP pressure means the ask side is concentrated higher, suggesting room for price appreciation.

2.3.5 Level-1 Concentrations

$$\text{bid_concentration} = \frac{\text{bid_qty}_1}{\text{mean}(\text{bid_qty}_{1..5})}$$

$$\text{ask_concentration} = \frac{\text{ask_qty}_1}{\text{mean}(\text{ask_qty}_{1..5})}$$

Why: High bid concentration means most buying interest is concentrated right at the top of the book. This is bullish — buyers are aggressive. Low concentration means buying is spread out, signaling passive accumulation.

2.3.6 Fill Price (Market Impact)

$$\text{fill_price_buy} = \frac{\sum_{i=1}^5 \min(\text{ask_qty}_i, \text{remaining}) \times \text{ask_price}_i}{500}$$

Why: Uses a 500-share simulated market order to walk the ask book and compute the actual average execution price. This is more realistic than best ask alone. When `fill_price_buy` is significantly above mid price, it signals thin ask-side liquidity and potential for sharp upward moves.

2.4 Group 3 — Support and Resistance Features

$$\text{support_price} = \text{bid_price at } \arg \max_i \text{bid_qty}_i$$

$$\text{resistance_price} = \text{ask_price at } \arg \max_i \text{ask_qty}_i$$

Why: Classical support and resistance is drawn on charts based on historical price levels. That is lagging and subjective. These features identify support as the live price level where the largest buy order is currently sitting. Price approaching a large bid wall behaves differently than price in open space — the model needs to know this difference.

Rolling strength (how sticky the wall is):

$$\text{support_strength} = \text{rolling_mean}(\text{bid_qty at support, } w = 50)$$

2.5 Group 4 — Multi-Scale Rolling Features

Five metrics computed at five tick windows $w \in \{1, 5, 15, 30, 60\}$, giving $5 \times 5 = 25$ features:

Feature	Formula and Rationale
<code>realized_vol_{w}t</code>	$\text{std}(\text{mid_price}, w)$ — Volatility is the most important regime signal. It spikes distinctly in crashes and compresses in trends. Using five windows gives full temporal resolution.
<code>rolling_return_{w}t</code>	$(\text{mid}_t - \text{mid}_{t-w}) / \text{mid}_{t-w}$ — Captures directional momentum at five timescales simultaneously. Positive sustained returns indicate a bullish regime at that scale.
<code>obi_mean_{w}t</code>	$\text{mean}(\text{obi}, w)$ — Rolling OBI mean captures the sustained directional bias, filtering out tick-by-tick noise in the raw OBI signal.
<code>spread_vol_{w}t</code>	$\text{std}(\text{spread}, w)$ — Spread volatility is a secondary regime indicator. Spreads and volume together define regimes based on liquidity and volatility conditions.
<code>volume_spike_{w}t</code>	$\sum(\text{volume}, w) - \text{mean}(\text{volume}, w)$ — Spike in volume often precedes price movement. Captures abnormal activity at each timescale.

Table 2.1: Multi-scale rolling features

Why five windows: Market regimes operate at different time horizons simultaneously. A 60-tick window captures the macro trend. A 5-tick window captures current momentum. A 1-tick window captures microstructure noise level. Using only one window means the model is blind to everything happening at other scales.

2.6 Feature Summary

Group	Count	Features
Price	5	<code>mid_price</code> , <code>spread</code> , <code>spread_pct</code> , <code>ltp_mid_delta</code> , <code>vwmp</code>
Order Book	11	<code>obi_l1</code> , <code>obi</code> , <code>depth_ratio</code> , <code>ask_depth</code> , <code>bid_depth</code> , <code>vwap_pressure</code> , <code>bid_conc</code> ,
Support/Resistance	6	<code>support_price</code> , <code>resistance_price</code> , <code>dist_to_support</code> , <code>dist_to_resistance</code> , <code>supp</code>
Multi-scale Rolling	25	5 metrics $\times$ 5 windows
Total	46	

Table 2.2: Feature groups and counts

Quality check results on real data:

- Null rate: 0.00% across all 46 features
- OBI range: $[-0.977, +0.915]$ (within expected $[-1, +1]$)
- Spread ≤ 0 : 0 rows
- Fill price buy $\geq$ LTP: 97.2% of rows (correct by definition)

2.7 Target Engineering

$$\text{target}_t = \mathbb{I}[\text{mid_price}_{t+30} > \text{mid_price}_t]$$

Binary classification: 1 = price goes up in 30 ticks, 0 = price goes down.

Why 30-tick horizon: Below 10 ticks you are predicting noise — autocorrelation of mid-price returns at sub-10-tick scale is near zero. Above 100 ticks the OBI signal decays and microstructure features lose predictive power. 30 ticks is the empirically supported sweet spot in LOB forecasting literature.

Target distribution on RELIANCE-EQ: UP = 15,183 (43.74%), DOWN = 19,530 (56.26%). Slight bearish skew on this particular trading day.

3. Regime Detection

3.1 Why Regime Detection

A single model trained on all market conditions learns the average of all regimes, which means it performs poorly in each specific one. A bullish model should weight OBI and momentum features heavily. A volatile model should weight spread and realized volatility features heavily. A model that does not know which condition it is in will average across all three and underperform in each.

3.2 Step 1 — Gaussian Mixture Model (GMM)

3.2.1 What It Is

A Gaussian Mixture Model (GMM) is an unsupervised probabilistic model that assumes data is generated from a mixture of K Gaussian distributions. It finds natural clusters in the feature space without requiring labeled data.

For $K = 3$ regimes, the model learns:

$$p(\mathbf{x}) = \sum_{k=1}^3 \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

where π_k are mixture weights, $\boldsymbol{\mu}_k$ are cluster means, and $\boldsymbol{\Sigma}_k$ are covariance matrices.

3.2.2 Why GMM Must Come First

The Adaptive Random Forest (ARF) used in Step 2 is a *supervised* classifier. Supervised means it needs labels to train on. You do not have regime labels — nobody gives them to you. GMM generates them first without needing any ground truth. Without GMM first, ARF has nothing to learn from.

3.2.3 Input Features

$$\mathbf{x} = [\text{realized_vol_60t}, \text{rolling_return_60t}]$$

60-tick windows because they capture the macro trend and volatility state that defines the regime, not tick-by-tick noise.

3.2.4 Regime Assignment

After fitting, cluster characteristics are computed and clusters are named:

- Highest realized volatility cluster → **Volatile** (regime 2)

- Most negative rolling return among remaining → **Bearish** (regime 1)
- Remaining cluster → **Bullish** (regime 0)

3.2.5 Why Not HMM Instead of GMM

A Hidden Markov Model (HMM) adds temporal dependency between states — the current regime depends on the previous regime. This is theoretically appealing but has two practical problems in this setting:

1. HMM training requires the Baum-Welch algorithm which does not scale well to 34,000 rows with multiple features.
2. The temporal dependency assumption is violated when regime changes happen suddenly (news events, large orders). GMM's independence assumption is actually more robust to sudden shifts.
3. HMM is not trivially online — it needs to be re-run on a rolling window. GMM output is instantly passed to ARF which handles the temporal adaptation.

3.3 Step 2 — Adaptive Random Forest (ARF)

3.3.1 What It Is

ARF is an ensemble of Hoeffding Trees (see Chapter 4) with two key additions:

1. **Internal drift detection:** Each tree monitors its own error rate using ADWIN. When drift is detected, the weakest tree is replaced automatically.
2. **Online learning:** The entire forest updates on every single tick via `learn_one()`, requiring no retraining.

3.3.2 Why ARF for Real-Time Regime Detection

Once GMM gives regime labels on historical data, we need something that can detect regime in real time on new ticks without retraining from scratch. ARF does this because:

- Truly online — updates per tick
- Non-linear — captures complex regime boundaries that a linear model misses
- Self-replacing — when market regime shifts in a way that makes its trees unreliable, it automatically replaces the weakest one
- No manual monitoring required

3.3.3 Input Features for ARF

$\mathbf{x}_{\text{ARF}} = [\text{realized_vol_60t}, \text{realized_vol_15t}, \text{rolling_return_60t}, \text{rolling_return_15t}, \text{obi_mean_60t}, \text{spread}]$

Both 15t and 60t windows are included to give the ARF multi-scale temporal context. A fast volatility spike (15t) within a slow downtrend (60t) is a different regime than a fast spike within a slow uptrend.

3.3.4 Why Not K-Means Instead of GMM

K-Means assigns hard cluster boundaries using Euclidean distance. This has two problems for financial data:

1. Financial regime features (volatility, return) are not spherically distributed. GMM uses full covariance matrices and fits ellipsoidal clusters, which matches the actual data geometry.
2. K-Means cannot produce soft regime probabilities. GMM's posterior $P(\text{regime} = k \mid \mathbf{x})$ can be used as a continuous regime signal, not just a hard label.

4. Online Forecasting Models

4.1 Design Principle

Three models with fundamentally different inductive biases run in parallel on every tick. Each model has a different assumption about how the world works:

Model	Assumption	When It Wins
FTRL	Linear relationship, features have varying importance that changes over time	Trending regimes where OBI and momentum signals are consistent
Hoeffding Tree	Non-linear decision boundaries, feature interactions matter	Regime boundary conditions, when single features alone are not enough
PA Classifier	Update aggressively on mistakes, do nothing on correct predictions	Low-volatility regimes where signal is weak and conservative updating prevents noise chasing

Table 4.1: Three forecasting models and their inductive biases

One model alone has a blind spot. Three models with different strengths cover each other's weaknesses.

4.2 Model 1 — FTRL (Follow The Regularized Leader)

4.2.1 Algorithm

FTRL is Google's production algorithm for real-time prediction at billion-scale (originally for ad click prediction). The update rule is:

$$w_{t+1} = \arg \min_w \left[\sum_{s=1}^t \ell_s(w) + \frac{1}{2} \sum_j (\lambda_1 |w_j| + \lambda_2 w_j^2) \right]$$

The key innovation is per-feature adaptive learning rates. The learning rate for feature j at time t is:

$$\eta_{t,j} = \frac{\alpha}{\beta + \sqrt{\sum_{s=1}^t g_{s,j}^2}}$$

where $g_{s,j}$ is the gradient with respect to feature j at step s .

4.2.2 Why FTRL Over Standard SGD

Standard SGD uses the same learning rate for all features. FTRL adapts per-feature. In this application:

- OBI has high variance and noisy gradients $\rightarrow$ FTRL gives it a smaller learning rate automatically
- Rolling Sharpe (ratio of return to volatility) is stable when present $\rightarrow$ FTRL trusts it more aggressively
- Features that become irrelevant in certain regimes get their learning rates suppressed automatically

L1 regularization in FTRL also produces sparse weight vectors, effectively doing automatic feature selection on the fly.

4.2.3 Why FTRL Over Passive-Aggressive for Trending Regimes

In trending regimes, OBI and momentum signals are consistently informative. FTRL's continuous updates (on every tick, not just mistakes) build up a stronger signal in this case. PA only updates on mistakes, which means in a clear trend where it is correct 70% of the time, it only updates 30% of the time — slowly adapting to the trend.

4.3 Model 2 — Hoeffding Tree (VFDT)

4.3.1 Algorithm

A Hoeffding Tree (Very Fast Decision Tree) builds incrementally from a stream. The key innovation is the Hoeffding bound: rather than splitting a node immediately when one feature appears better than another, it waits until statistical confidence is sufficient.

The Hoeffding bound guarantees: with probability $1 - \delta$, the true best feature is identified after seeing n examples if:

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

where R is the range of the split criterion and $\epsilon < \Delta \tilde{G}$ (the difference between the two best feature scores).

4.3.2 Why Non-Linear for Regime Boundaries

Linear models like FTRL and PA assume the decision boundary between UP and DOWN is a hyperplane in feature space. At regime transitions, this assumption breaks down. For example:

- In a bull regime: $\text{OBI} > 0.3$ predicts UP
- In a volatile regime: $\text{OBI} > 0.3$ AND $\text{spread_vol} < 0.05$ predicts UP; $\text{OBI} > 0.3$ AND $\text{spread_vol} > 0.05$ predicts NEUTRAL

The Hoeffding Tree captures this interaction automatically. The linear models cannot.

4.3.3 Grace Period Warning

The Hoeffding Tree returns `None` for the first 50 predictions (`grace_period=50`) while it accumulates sufficient statistics to make a first split. The ensemble handles `None` predictions by defaulting to 0. After the grace period it begins contributing normally.

4.4 Model 3 — Passive-Aggressive Classifier (PA)

4.4.1 Algorithm

The PA-I update rule is:

$$w_{t+1} = w_t + \tau_t y_t x_t$$

where:

$$\tau_t = \frac{\max(0, 1 - y_t \langle w_t, x_t \rangle)}{\|x_t\|^2 + \frac{1}{2C}}$$

When the prediction is correct and the margin is sufficient ($y_t \langle w_t, x_t \rangle \geq 1$), $\tau_t = 0$ and no update happens. When a mistake is made or the margin is insufficient, the update is proportional to the loss.

4.4.2 Why PA for Low-Volatility Regimes

In a low-volatility regime, the market signal is weak and predictions are often close to the decision boundary. Standard online learning models update on every tick and will chase the noise, oscillating their weights rapidly.

PA is structurally conservative: it only updates when wrong or when the margin is insufficient. In a low-vol regime where 70% of predictions are correct (but barely), PA stabilizes its weights and maintains consistent predictions. FTRL in the same setting would jitter weights on every tick.

4.4.3 Why PA Achieved 96% Accuracy and What That Means

PA alone achieved 96.01% accuracy on the test set. This number is suspiciously high and requires honest explanation:

Important

NSE tick data has strong short-term autocorrelation within a single trading day. Mid price at tick t is highly correlated with mid price at tick $t+30$ within the same intraday session. PA's aggressive updating on a single day of data effectively memorizes this autocorrelation pattern. On a new trading day with different intraday dynamics, this accuracy will be significantly lower. This is not a generalizable result — it is overfitting to intraday serial correlation in a one-day dataset. Day 2 validation confirmed this: PA alone achieved 95.01% with online learning on the new day, consistent with the same autocorrelation effect, while frozen inference dropped to 58%.

4.5 Input Features for All Three Forecasters

Feature Name	Source Column
obi_1t	obi_mean_1t
obi_5t	obi_mean_5t
obi_15t	obi_mean_15t
spread_5t	spread_vol_5t
spread_15t	spread_vol_15t
ltp_mid_delta	ltp_mid_delta
vwmp	vwmp
rolling_return_5t	rolling_return_5t
rolling_return_15t	rolling_return_15t
dist_to_support	dist_to_support
dist_to_resistance	dist_to_resistance
support_strength	support_strength
resistance_strength	resistance_strength

Table 4.2: Forecaster input features (13 total)

5. Dynamic Ensemble and Drift Detection

5.1 Why Dynamic Weighting and Not Fixed

If weights are fixed permanently — FTRL always gets 0.6 in a bullish regime — you are assuming your initial assumption about which model is best never changes. Markets evolve intraday. The model that was best in the first hour may be worst in the third hour. Dynamic weighting based on rolling accuracy means the ensemble is continuously re-evaluating which model deserves trust right now.

5.2 Two-Stage Weight Computation

5.2.1 Stage 1 — Regime Prior

Regime-based initial weights (prior belief about model suitability):

Regime	FTRL	Hoeffding	PA
Bullish (0)	0.6	0.3	0.1
Bearish (1)	0.3	0.5	0.2
Volatile (2)	0.2	0.3	0.5

Rationale: FTRL leads in trending regimes. Hoeffding leads in bearish regimes where non-linear boundaries matter. PA leads in volatile regimes where conservative updating prevents noise chasing.

5.2.2 Stage 2 — Rolling Accuracy Adjustment

Over the last 100 ticks, the rolling accuracy of each model is computed:

$$\hat{a}_m = \frac{\sum_{t-100}^t \mathbb{1}[\hat{y}_m^{(t)} = y^{(t)}]}{100}$$

Accuracy-normalized weights:

$$w_m^{\text{acc}} = \frac{\hat{a}_m}{\sum_{m'} \hat{a}_{m'}}$$

Final blended weights (50/50 blend):

$$w_m^{\text{final}} = \frac{0.5 \cdot w_m^{\text{regime}} + 0.5 \cdot w_m^{\text{acc}}}{\sum_{m'} (0.5 \cdot w_{m'}^{\text{regime}} + 0.5 \cdot w_{m'}^{\text{acc}})}$$

5.2.3 Final Prediction

$$\text{score} = w_{\text{ftrl}} \cdot \hat{y}_{\text{ftrl}} + w_{\text{hoff}} \cdot \hat{y}_{\text{hoff}} + w_{\text{pa}} \cdot \hat{y}_{\text{pa}}$$

$$\hat{y}_{\text{final}} = \mathbb{I}[\text{score} \geq 0.5]$$

5.3 ADWIN Drift Detection

5.3.1 What ADWIN Does

ADWIN (Adaptive Windowing) maintains a variable-size window of recent error observations. It uses a statistical test to detect when the error rate in the recent half of the window is significantly different from the older half. When detected, it shrinks the window and signals drift.

The test condition for splitting window at position i :

$$|\hat{\mu}_0 - \hat{\mu}_1| \geq \epsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \ln \frac{4n}{\delta}}$$

where m is the harmonic mean of the two sub-window sizes, n is the total window size, and δ is the confidence parameter.

5.3.2 Why Not Reset Models on Drift

The naive response to drift is to reset all model weights to zero. This is wrong.

Resetting throws away everything the model learned. If FTRL learned that OBI above 0.4 predicts upward movement with 70% reliability, resetting deletes that knowledge. Recovery from zero takes hundreds of ticks during which predictions are unreliable.

Increasing the learning rate instead tells the model "something changed, weight new observations more heavily" while keeping prior knowledge as a starting point. The system adapts fast without the blind period.

5.3.3 On Drift: Boost, Not Reset

When ADWIN detects drift:

- FTRL α increases from 0.1 to 0.3 (faster adaptation)
- PA C increases from 0.1 to 0.5 (more aggressive on mistakes)
- ARF replaces its weakest tree automatically (internal drift handling)

7 drift events were detected in the Day 1 test set. 9 drift events were detected on Day 2 (10,250 ticks), confirming the system adapts automatically across sessions without human intervention.

6. Algorithms Considered But Not Used

6.1 Models Evaluated and Rejected

6.1.1 Kalman Filter (considered for price regression)

What it is: A Bayesian filter that models price as a hidden state evolving over time, with ticks as noisy observations. Optimal for linear Gaussian systems.

Why rejected: The prediction target was changed to binary UP/DOWN direction rather than absolute price. Using Kalman Filter as a direction signal (thresholding the state estimate) is methodologically unsound — the filter is designed for continuous state estimation, not classification. PA Classifier is the correct tool for online binary classification.

Future use: Valid if the target becomes price return prediction rather than direction classification.

6.1.2 LinUCB Contextual Bandit (considered for model selection)

What it is: A bandit algorithm that learns which model (arm) to use based on the current feature context (regime). Balances exploration (trying less-used models) vs exploitation (using the proven best model).

Why not implemented: The dynamic weighted ensemble already handles model selection based on rolling accuracy, which is a simpler and more interpretable approach. LinUCB adds significant complexity for marginal gain when the ensemble weights already adapt continuously.

Future use: LinUCB becomes the superior choice when running many more parallel models (e.g., 10+ specialized sub-models), where the combinatorial space of model selection is too large for simple weighting.

6.1.3 HMM (considered for regime detection)

What it is: Hidden Markov Model with temporal dependency between regime states. Standard in quant finance literature.

Why GMM was chosen instead:

1. HMM is not online — requires the Baum-Welch EM algorithm which runs on the full sequence
2. HMM's temporal dependency assumption is violated by sudden regime changes (news events)
3. GMM + ARF achieves the same goal with cleaner separation of concerns: GMM for unsupervised label generation, ARF for online detection

6.1.4 DeepLOB / TCN (considered for sequence modeling)

What it is: CNN+LSTM (DeepLOB) or Temporal Convolutional Network applied to raw LOB snapshot sequences. State-of-the-art for mid-price direction prediction in academic literature.

Why rejected:

1. Requires 5–20 days of data minimum for meaningful training. One day of data is insufficient.
2. Inference latency is 1–5ms per prediction. Acceptable for 100–500ms WebSocket ticks but eliminates any latency advantage.
3. Not truly online — requires batch fine-tuning, not per-tick updates.
4. The added complexity is not justified when feature-engineered models with domain knowledge achieve comparable accuracy on this dataset.

6.1.5 LightGBM / XGBoost / CatBoost (tested as offline benchmarks)

Results from `algorithm_comparison.ipynb`:

Algorithm	H=50	H=100	H=200	H=300
LightGBM	0.3148	0.3025	0.3276	0.3032
XGBoost	0.3106	0.2997	0.3099	0.2963
CatBoost	0.2549	0.2890	0.2839	0.2690
RandomForest	0.2857	0.3189	0.3251	0.2883
LogisticRegression	0.3060	0.2787	0.2445	0.2483

Table 6.1: Macro F1 scores on 3-class direction prediction (old pipeline)

Why rejected for the live system:

1. Not online — cannot update per tick without full retraining
2. Macro F1 ≈ 0.33 on the old 3-class formulation is barely above random chance (random = 0.33 for 3 classes)
3. The new binary formulation with online models achieves 85% accuracy vs 41% naive baseline, far superior

LightGBM remains useful as an offline benchmark and for feature importance analysis.

6.1.6 AROW (Adaptive Regularization of Weights)

What it is: Maintains a full covariance matrix over weight uncertainty, not just point estimates. Updates both the mean and confidence of each weight simultaneously.

Why not implemented: The three chosen models (FTRL, Hoeffding, PA) already cover the key inductive biases. AROW would add a fourth linear model that largely overlaps with PA in behavior. The complexity is not justified.

6.1.7 Ridge / Lasso (tested as price regressors)

Results: Ridge at $H=50$ achieved $MAE = 0.3322$ Rs vs baseline 0.3366 Rs — only 1.3% improvement over predicting zero price change. Tree-based regressors were catastrophically bad ($MAE \approx 18$ Rs) because they overfit to absolute price levels which are non-stationary.

Why rejected: The target was changed to binary direction instead of absolute price, which is more tractable and more practically useful for a trading signal.

7. Results

7.1 Key Results — Day 1 (May 13 2026)

Naive baseline (always UP)		41.15%
Ensemble accuracy		85.04%
Improvement		+43.89 percentage points

7.2 Per-Regime Accuracy — Day 1

Regime	Ticks	Accuracy	Notes
Bullish	0	—	Not observed on this trading day
Bearish	1,119	82.75%	
Volatile	5,824	85.47%	

The absence of Bullish regime is expected — May 13 2026 RELIANCE-EQ was a down/choppy day. With more data across multiple days, all three regimes will be populated.

7.3 Model Comparison — Day 1

Model	Accuracy
Naive (always UP)	41.15%
Hoeffding Tree alone	60.90%
FTRL alone	73.70%
Ensemble	85.04%
PA alone	96.01%*

*See Section 4.3 for explanation of this number

7.4 Latency

Metric	Value
Mean	315,336 ns (0.315 ms)
p50	289,900 ns
p99	643,164 ns
Max	971,900 ns
WebSocket tick interval	100–500 ms

All predictions complete well within the WebSocket tick interval. The system never falls behind the tick stream.

7.5 Drift Events — Day 1

7 drift events detected by ADWIN across the test set (6,943 ticks). Each event triggered a learning rate boost. The system adapted automatically without human intervention.

7.6 Day 2 Generalisation Test (May 14 2026)

The ensemble was tested on a completely separate trading session — 10,250 ticks from May 14 2026 (09:08–10:37 IST) — to validate that Day 1 accuracy was not an artefact of intraday autocorrelation alone. The system ran in online learning mode: models loaded from Day 1 and continued updating on the new day’s stream.

	Day 1	Day 2
Naive baseline	41.15%	41.88%
Ensemble accuracy	85.04%	83.41%
Accuracy drop	–1.63 percentage points	

The model generalises. Only 1.63 percentage points were lost moving to a completely unseen trading session. Both UP and DOWN classes were predicted with balanced precision and recall — unlike Day 1’s frozen inference failure, which collapsed all predictions to DOWN.

Day 2 Classification Report

Class	Precision	Recall	F1	Support
DOWN	0.8401	0.8827	0.8608	5,957
UP	0.8249	0.7668	0.7948	4,293
Accuracy		0.8341		10,250
Macro avg	0.8325	0.8247	0.8278	10,250

Day 2 Per-Regime Accuracy

Regime	Ticks	Accuracy
Bearish	2,548	82.73%
Volatile	7,702	83.64%

Day 2 Single Model vs Ensemble

Model	Accuracy
FTRL alone	70.58%
Hoeffding alone	65.92%
PA alone	95.01%*
Ensemble	83.41%

*PA memorises intraday autocorrelation; see Section 4.3

Day 2 Latency and Drift

Mean inference latency dropped to 151,912 ns (0.152 ms) on Day 2, down from 0.315 ms on Day 1. 9 drift events were detected and handled automatically via learning rate boosts, with no human intervention.

Note

Online learning is required for cross-day inference. Running `--no_learn` (frozen weights) on Day 2 produced 58.12% accuracy — identical to the always-DOWN naive baseline. The root cause is that the forecasters require online updates to adapt the rolling feature context to a new session’s intraday dynamics. The updated `test_new_data.py` implements a 60-row rolling warmup to address cold-start rolling feature initialisation.

8. System Design Decisions

8.1 Why Time-Ordered Train/Test Split

Financial data is non-stationary and temporally autocorrelated. A random split leaks future data into training. The morning of the trading day would be in the test set while the afternoon would be in training, meaning the model has "seen the future." This inflates accuracy by 5–15 percentage points and produces numbers that are entirely fictitious in live trading.

Time-ordered split: first 80% of each symbol's rows for training, last 20% for testing. This is the only evaluation that reflects real-world performance.

8.2 Why Binary Target Instead of 3-Class

The original pipeline used 3-class direction labels: UP (+1), NEUTRAL (0), DOWN (-1) with a 0.05% threshold. At H=50, only 19% of rows were UP or DOWN — 81% were NEUTRAL. This extreme class imbalance made the minority classes (the actionable predictions) nearly impossible to predict.

Binary UP/DOWN removes the NEUTRAL class. Every tick has a prediction. The resulting 44%/56% split is tractable for all three online models.

8.3 Why Not Reset Models on Drift

Resetting throws away all learned weights. If FTRL has learned that OBI above 0.4 predicts upward movement with 70% reliability, resetting deletes that knowledge. Recovery from zero takes hundreds of ticks during which predictions are unreliable. Increasing the learning rate instead tells the model to weight new observations more heavily while keeping prior knowledge as a starting point.

8.4 Why Python and Not C++

The existing production C++ engine (`pa_engine.cpp`) achieves sub-microsecond inference via AVX2 SIMD. Python inference in this system averages 315 μ s on Day 1 and 152 μ s on Day 2. For a WebSocket feed with 100–500ms tick intervals, Python latency is entirely acceptable.

C++ becomes necessary for co-location deployments where tick-to-order latency must be under 10 μ s. This is future work.

9. Limitations and Future Work

9.1 Honest Limitations

1. **Two days of data:** The system has been validated on two consecutive trading days (May 13–14 2026). GMM was fitted on Day 1 only and has not encountered a Bullish regime. Longer-term generalisation across varied market conditions remains unproven.
2. **PA accuracy reflects autocorrelation:** PA alone achieves 95–96% in online learning mode by memorising intraday serial correlation. Frozen inference on a new day drops to 58%. The ensemble (83–85%) is the meaningful number, not PA in isolation.
3. **No Bullish regime observed:** Both May 13 and May 14 2026 were bearish/volatile sessions for RELIANCE-EQ. The Bullish model path has never been exercised. Ensemble weights for Bullish regime (FTRL=0.6) are untested assumptions.
4. **Online learning required for cross-day inference:** Frozen (`--no_learn`) inference on a new day fails because forecasters need to adapt to the new session’s rolling feature distribution. The warmup fix in `test_new_data.py` addresses cold-start initialisation but not the fundamental need for continued adaptation.
5. **5-level book only:** WebSocket Mode 3 provides only 5 bid and 5 ask levels. Institutional players with 20+ level access have an information advantage.
6. **No alternative data:** No news, no FII/DII flow, no options data, no sentiment. The model is purely technical.
7. **Single symbol:** Trained and tested on RELIANCE-EQ only. Generalization to other symbols is unverified.

9.2 Future Work (Priority Order)

1. Collect 20+ days of data — expose all three regimes to GMM, validate model generalisation across varied market conditions
2. Implement walk-forward cross-validation — expanding window retraining to get robust out-of-sample estimates
3. LinUCB contextual bandit — replace hardcoded regime-to-weight mapping with a learned selector

4. Replace Python inference with C++ for co-location deployments (extend existing `pa_engine.cpp`)
5. Add signed order flow imbalance (OFI) and trade tick features (the pipeline currently only ingests L2 snapshots)
6. Multi-symbol generalisation test — train on RELIANCE, evaluate on TCS/HDFC to see whether the model captures general NSE microstructure or just one name’s quirks
7. 2-state HMM on (OBI, log-spread) to replace GMM for regime detection once sufficient data is available
8. Add PnL simulation with realistic transaction costs to convert directional accuracy into a tradeable signal assessment

10. How to Run

10.1 Dependencies

```
1 pip install pandas numpy river scikit-learn hmmlearn \
2     openpyxl pyarrow streamlit plotly
```

10.2 Full Pipeline (One Command)

```
1 python main.py --input data/orderbook.csv.xlsx
```

This single command runs all steps in order: feature engineering, regime detection, and backtesting. It also saves `reliance_features.parquet` which is required for Day 2 testing.

10.3 Step by Step

```
1 # Step 1: Feature engineering
2 # Produces reliance_features.parquet      required for Day 2
   warmup
3 python feature_engine.py \
4     --input data/orderbook.csv.xlsx \
5     --output reliance_features.parquet
6
7 # Step 2: Regime detection
8 python regime/regime_detector_v2.py \
9     --parquet reliance_features.parquet \
10    --output regime/regime_model.pkl
11
12 # Step 3: Backtest
13 python backtest.py \
14     --parquet reliance_features_regime.parquet \
15     --regime_model regime/regime_model.pkl
```

10.4 Test on New Day's Data

Important

Run the full pipeline first. `test_new_data.py` requires `reliance.features.parquet` to exist in the project root. Without it, forecasters start cold and all models collapse to predicting one class (58% accuracy = naive baseline). Always run `python main.py --input data/orderbook.csv.xlsx` before testing on new data.

```

1 # Recommended: online learning mode
2 # Models adapt to the new day's stream as they would in live
   trading.
3 # Requires reliance_features.parquet for forecaster warmup.
4 python test_new_data.py --input data/new_day.xlsx
5
6 # Inference only: models frozen at Day 1 weights.
7 # NOTE: accuracy will degrade significantly without online
   adaptation.
8 # Only use this to measure frozen generalisation, not live
   performance.
9 python test_new_data.py --input data/new_day.xlsx --no_learn

```

Expected results with correct warmup and online learning:

Mode	Expected Accuracy	Notes
(default) online learning	80–85%	Models adapt per tick
<code>--no_learn</code> frozen	~58–65%	No intraday adaptation

10.5 Dashboard

```

1 streamlit run dashboard.py

```

10.6 Live API Streaming

```

1 # Requires .env credentials and market hours (Mon-Fri
   09:15-15:30 IST)
2 python regime/live_engine.py

```

Note

Live streaming requires a valid `.env` file containing Angel One SmartAPI credentials. Never commit `.env` to version control.

10.7 Project File Structure

```

1 PROJECT/
2 |-- data/
3 |   |-- orderbook.csv.xlsx      raw NSE order book data (Day
4 |   |-- new_day.xlsx           Day 2 data for out-of-sample
   |   testing
5 |-- regime/
6 |   |-- __init__.py
7 |   |-- regime_detector_v2.py   GMM + ARF regime detection
8 |   |-- forecasters.py         FTRL + Hoeffding + PA
9 |   |-- ensemble.py            dynamic weights + ADWIN
10 |   |-- live_engine.py         live API streaming
11 |-- smartapi/                 Angel One API wrapper
12 |-- feature_engine.py         46 features from raw ticks
13 |-- backtest.py               full Day 1 evaluation
14 |-- test_new_data.py          out-of-sample Day 2 test
15 |-- main.py                   single entry point
16 |-- dashboard.py              Streamlit dashboard
17 +-- .env                       API credentials (never commit
   )
18
19 Generated at runtime:
20 |-- reliance_features.parquet   Day 1 features (needed for
   |   Day 2 warmup)
21 |-- reliance_features_regime.parquet features + regime labels
22 +-- regime/regime_detector.pkl   trained GMM + ARF model

```